MVC, controller et DTO avec Spring (cours 3)

I. Mougenot (isabelle.mougenot@umontpellier.fr)

Faculté des Sciences Université Montpellier

2026

Plan du cours

Focus sur les niveaux domaine et métier

1. pattern MVC et SPRING
2. Contrôleur et facade
3. Patterns DTO et DataMapper
4. Les contrôleurs Spring et leur rôle de façade
5. Cas d'usage et exemples de code

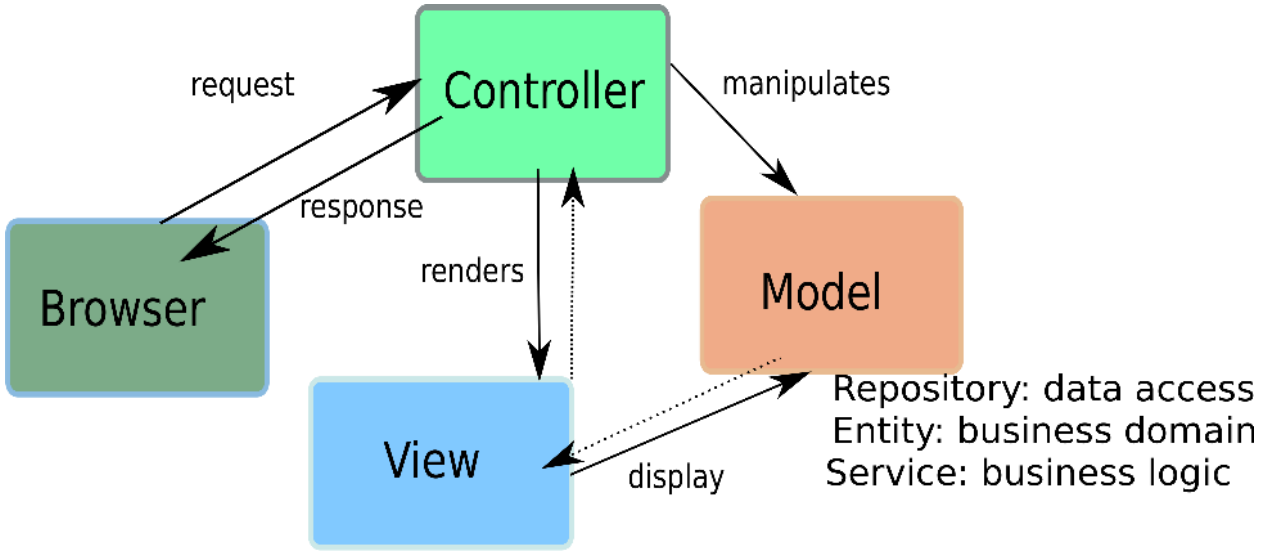
Large emprunt au cours de Chouki Tibermacine

UNIVERSITÉ DE MONTPELLIER

Pourquoi Spring MVC ?

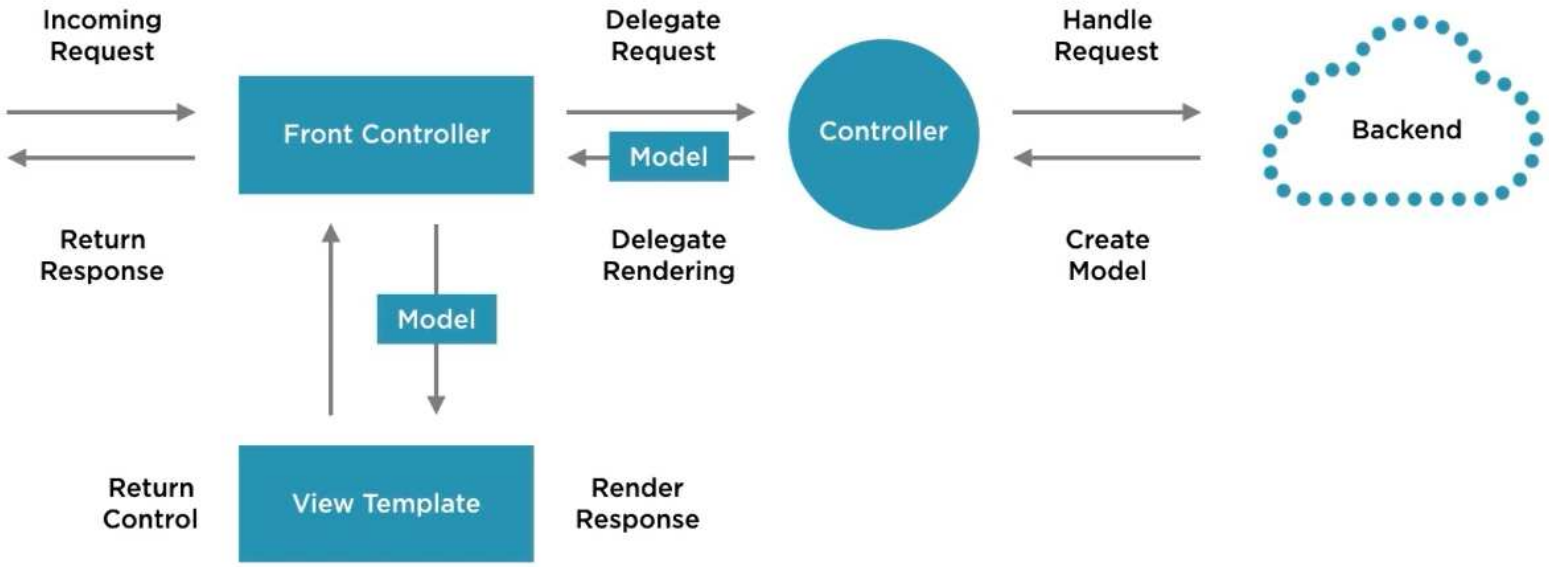
- Gérer la complexité d'applications d'entreprise, en séparant les rôles et en les répartissant sur différents composants (gérant le mapping des requêtes, la résolution des vues, la validation des modèles, ...)
- Programmation simplifiée par configuration (instanciation automatique, recherche d'instances, . . . revoir injection de dépendances)
- Besoin de développer des applications avec une API REST ou des applications SPA (*Single Page*)
- Besoin de concevoir des applications indépendamment d'une technologie de templates (HTML, JSP, ...) pour le front-end ou de persistance des données pour le back-end (JDBC, JPA, Hibernate, ...)

MVC avec Spring



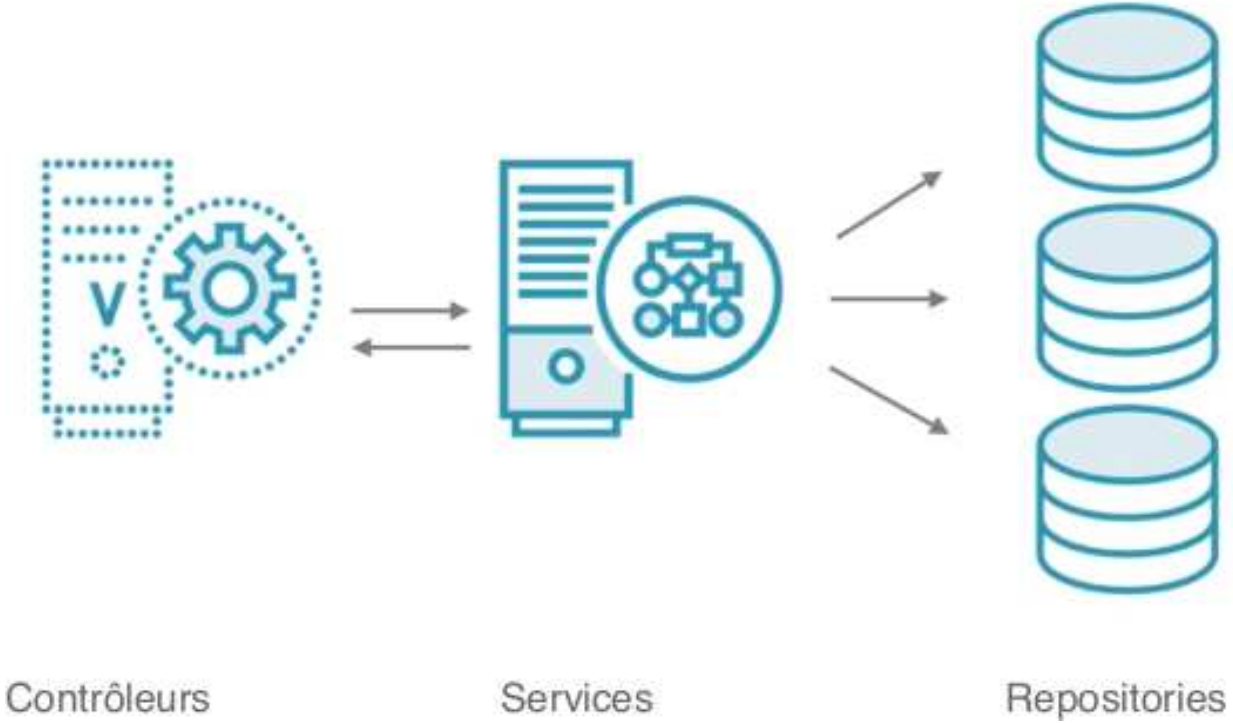
Le modèle dans Spring est subdivisé en différents niveaux

Cycle de vie d'une requête/réponse HTTP



ici Modèle : ensemble de données (map de clés/valeurs)

Des services derrière les contrôleurs



Positionnement d'un contrôleur

Un service du point de vue du client : tout ce qui se trouve au bout de la connexion HTTP

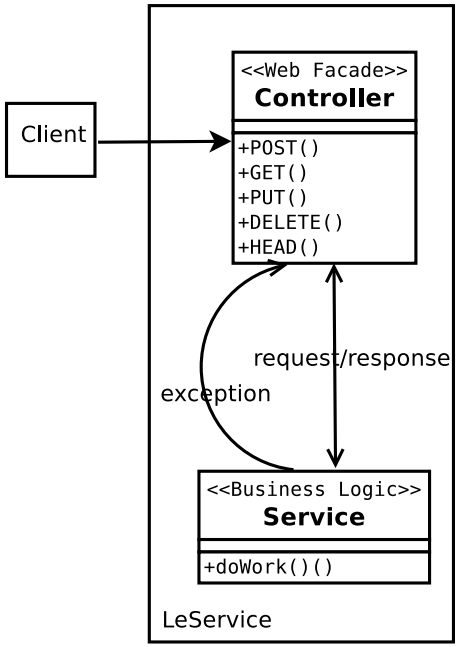


Figure: Perspective du client sur la notion de service

contrôleur : principalement une "façade web" interfaçant la logique métier avec le monde extérieur

Composant "Contrôleur" dans Spring MVC

- Classe annotée par `@Controller`
- Gère les requêtes/réponses
- N'embarque pas de logique métier
- Coordonne les services
- Gère les exceptions et le routage vers les vues

Composants "Service" et "Service" dans Spring MVC

Pour Service

- Classe annotée `@Service`
- Décrit les verbes/actions de l'application
- Là où réside la logique métier
- Garantit un état cohérent des objets métier
- Là où démarrent/s'arrêtent les transactions

Pour Repository

- Classe annotée `@Repository`
- Lieu des interactions avec les bases de données

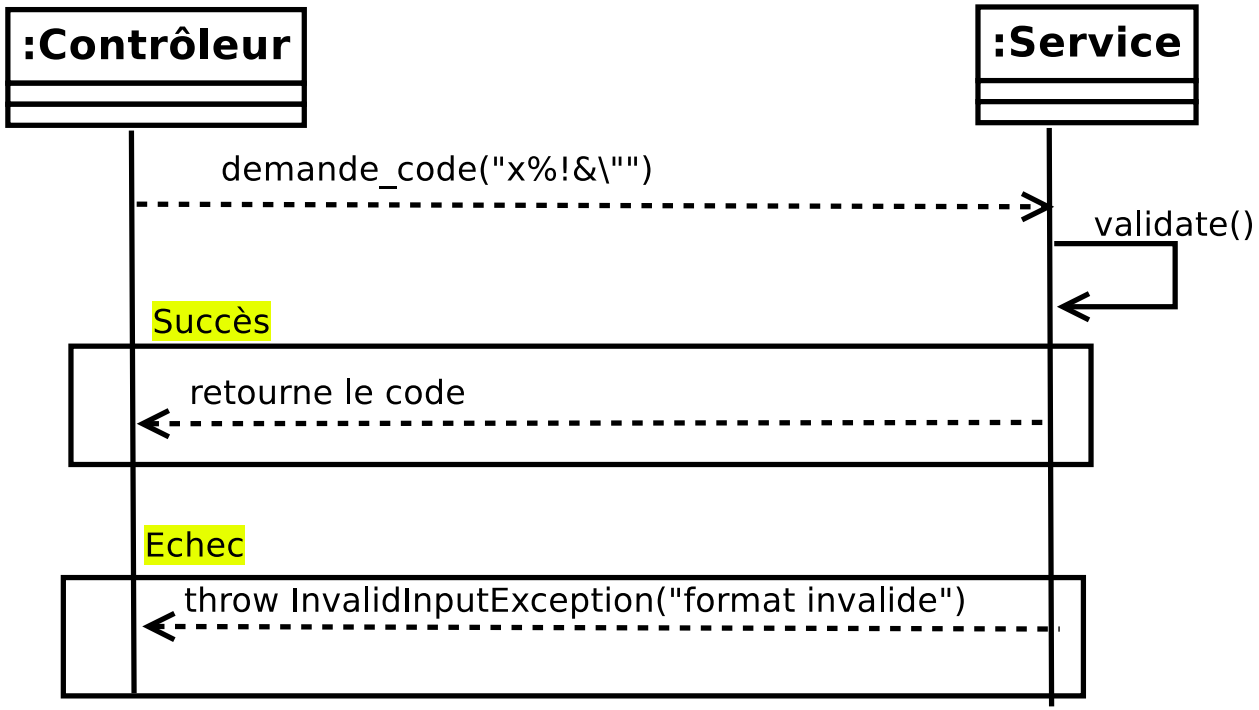
Que fait un contrôleur ? au cœur des interactions

oriente et dirige les flux de données entre entités

- Interprète et potentiellement transforme les requêtes
- Donne accès à la logique métier
- Détermine la vue ou le type de réponse
- Gère les exceptions

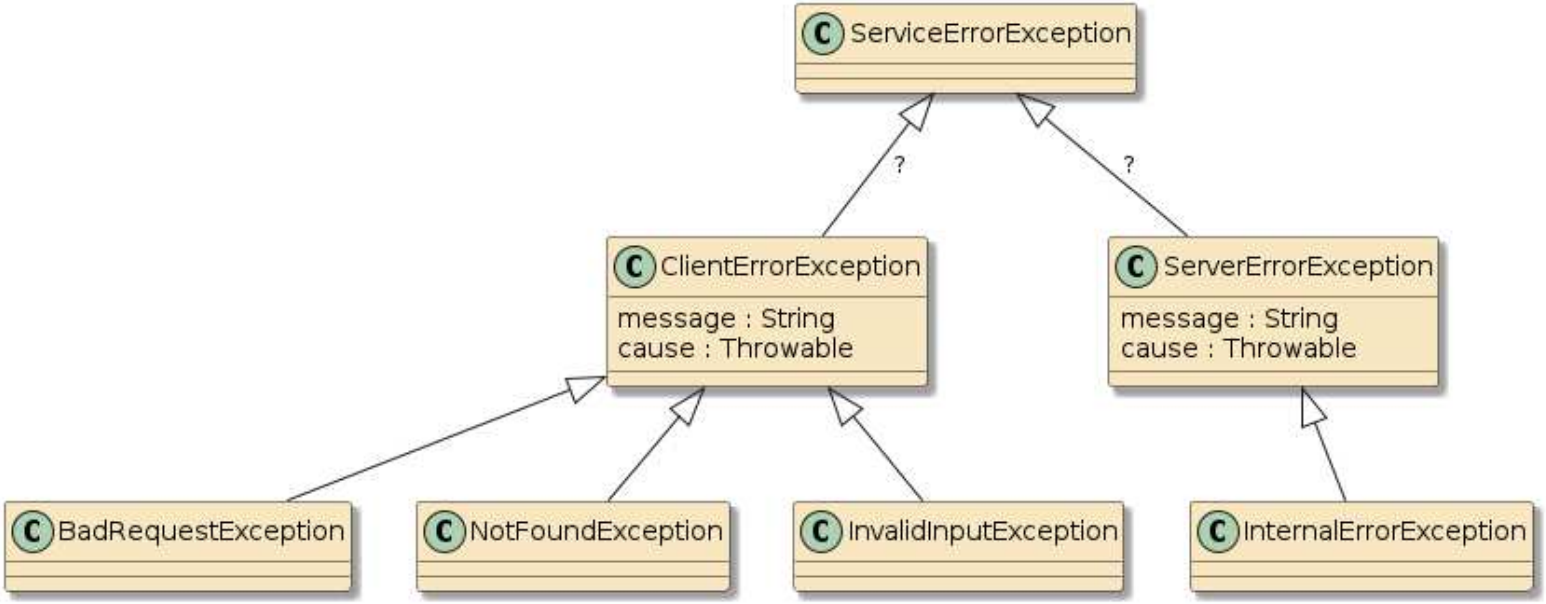
UNIVERSITÉ DE MONTPELLIER

Exemple élémentaire pour la gestion des exceptions



séparation claire des résultats attendus émanant de la logique métier et de la gestion des chemins d'erreur

Exemple élémentaire pour la gestion des exceptions



possibilité de certains choix de conception pour les exceptions
Le contrôleur peut intercepter ces exceptions via @ExceptionHandler et/ou @ControllerAdvice et renvoyer une réponse HTTP appropriée. (statut HTTP/ message JSON / Vue avec message)

Centralisation de la gestion d'erreurs

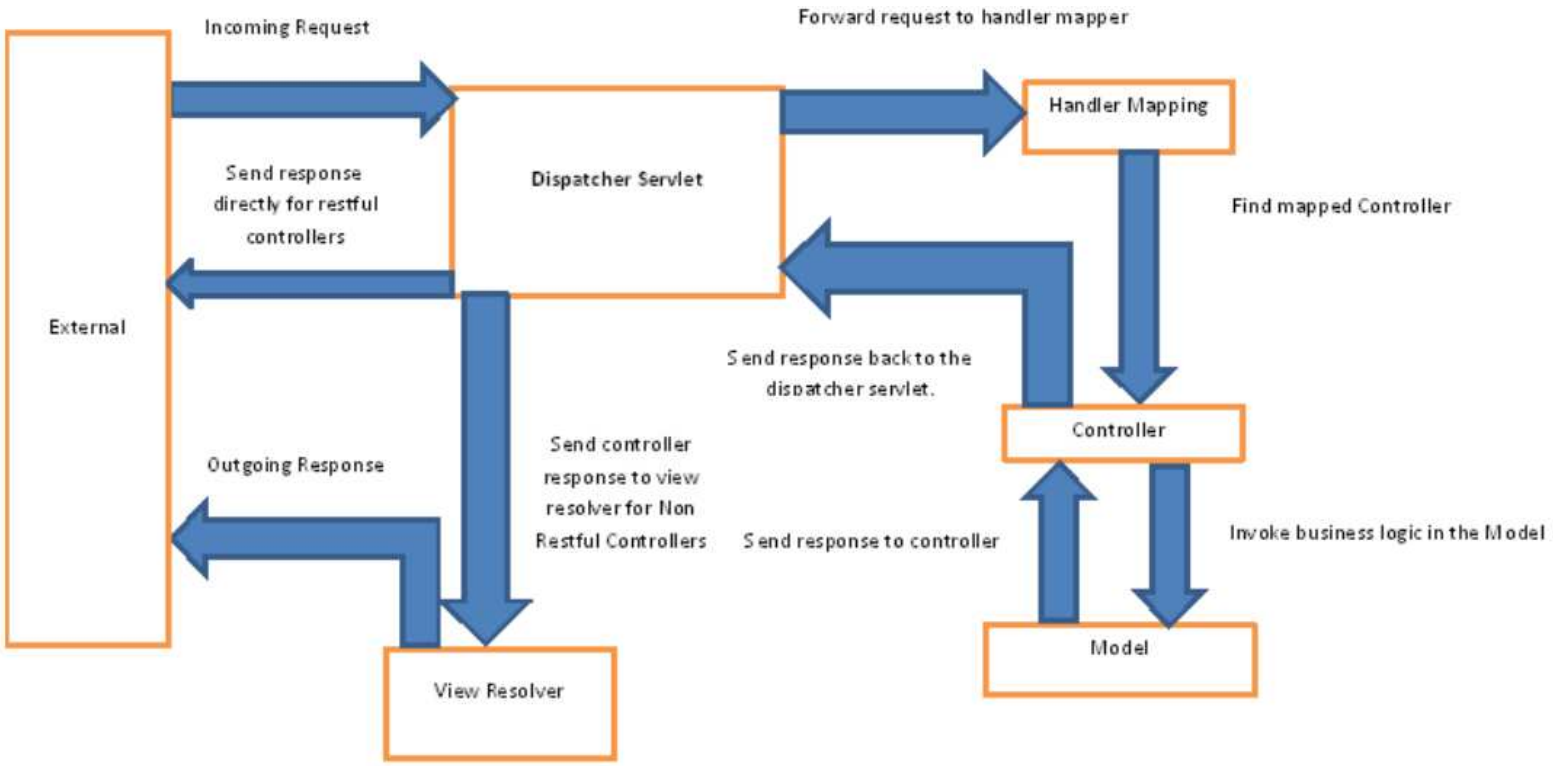
```
/* interception erreurs provenant des controllers */

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(CampusNotFoundException.class)
    public String handleCampusNotFound
      (CampusNotFoundException ex, Model model)
      {
        model.addAttribute("message", ex.getMessage());
        return "errors/campus_errors";
    }
}
```

Front Controller Pattern

- Spring MVC utilise le patron d'architecture *Front Controller* pour gérer le front avec les clients
- Ce rôle est joué par un objet particulier *Dispatcher Servlet* pour Spring
- Ses missions sont :
  1. dispatcher les requêtes vers des composants *Delegate* gérés par Spring (i.e. *Request Mapping*)
  2. résoudre les vues (*View Resolution*)
  3. gérer les exceptions (*Exception Handling*)

UNIVERSITÉ DE MONTPELLIER

Front Controller Pattern -suite-



Exemple élémentaire : contrôleur de vue

Focus sur les contrôleurs (@Controller) mobilisés au sein d'applicatifs Web "classiques" pour lesquels des vues vont être rendues au moyen de templates côté serveur (JSP ou Thymeleaf)

- l'accès à la racine de l'application renvoie à la vue index
- les vues sont placées par **convention** dans le sous-répertoire *templates* du répertoire *resources*
  1. L'annotation @RequestMapping met en relation ("map") la requête Web avec la méthode du contrôleur qu'elle vient décorer (par défaut méthode HTTP GET)
  2. La route est donnée au sein de cette annotation
  3. La sortie de la méthode est dirigée vers une vue au moyen du return

Contrôleur "accueil"

```
package Ex.control;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class IndexController {

    @RequestMapping("/index")
    String index(){
        return "index";
    }
}
```

Vue index.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head lang="en">
  <title>Universit&eacute; de Montpellier</title>

</head>
<body>
  <h1> Une vieille Universit&eacute; en Europe </h1>
  <p><img width="100px" th:src="@{images/UM_Logo.png}" /> </p>
</body>
</html
```

À noter un première contribution du moteur de templates rendu pour l'affichage de l'image UM_Logo.png (hébergée dans le sous-répertoire `images` du sous-repertoire `static` du répertoire `resources` du projet)

Annotations pour les contrôleurs de vues

- Annotations utilisées pour marquer les classes qui vont s'exécuter à la réception de requêtes HTTP
- Dans ces classes, on déclare les méthodes qui vont recevoir chaque type de requête (GET, POST, ...) pour une route donnée
- Ces méthodes seront annotées `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, ou avec l'annotation générique `@RequestMapping`
- Les routes sont indiquées dans ces annotations

Annotations pour les contrôleurs de vues -suite-

- Les méthodes des contrôleurs peuvent déclarer des paramètres
- Ces paramètres peuvent être rattachés directement à des données transmises par les vues (champs dans un formulaire, par exemple)
- Ces paramètres sont annotés avec par exemple `@ModelAttribute`

Exemple d'attribut de modèle

- Un objet (bean entity) de type `Campus` qui fait partie du modèle est passé en paramètre

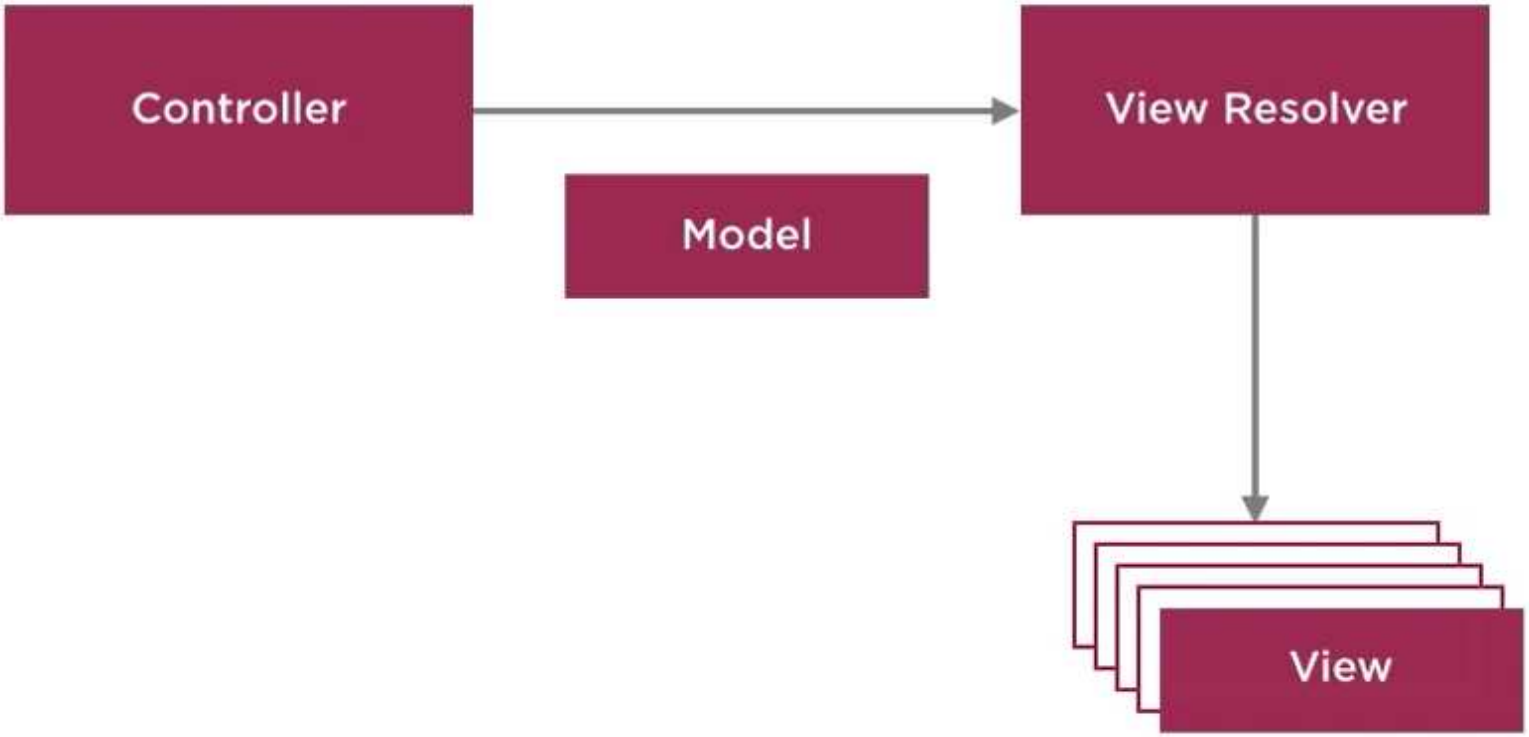
```
@RequestMapping(method = RequestMethod.GET)
public String setupForm(@ModelAttribute Campus campus) {
    // campus.setters...
    return "campus";
}
```

Mécanisme connu comme "liaison de données (data binding dans Spring MVC)

Lier les vues aux modèles (ici du JSP)

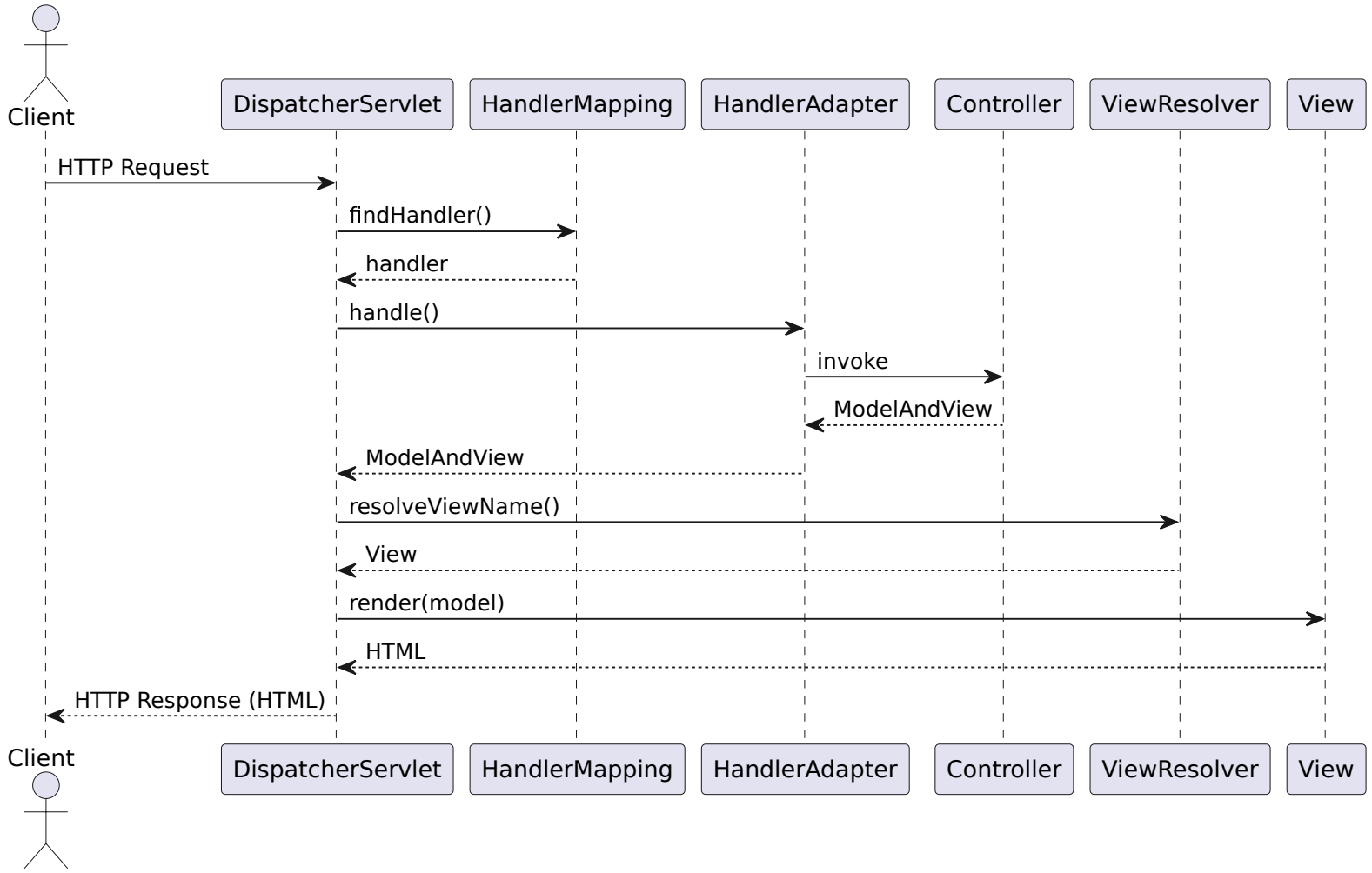
```
<%@ page contentType="text/html;charset=UTF-8"
         language="java" %>
<%@taglib prefix="form"
          uri="http://www.springframework.org/tags/form" %>
<html>
 <head><title>Ajouter un campus</title></head>
<body>
 <h2>campus</h2>
 <form:form modelAttribute="campus">
  <div> Nom :
  <form:input path="nomC"/>
   ... </div>
  <div> Ville :
  <form:input path="ville"/> ... </div>
</form:form>
</body>
</html>
```

View Resolver



- Un objet utilisé par le contrôleur pour choisir les vues. L'interface ViewResolver fournit le mapping entre les noms des vues et les vues elles même

Diagramme de séquences sur la chaîne d'activités



- Contrôleur MVC "classique"

Interaction avec le modèle

- interface `Model` (org.springframework.ui.Model) qui permet de fournir les attributs requis pour le rendu des vues. Le modèle est envisagé comme une Map (java.util.Map)
- interface `ModelMap` aussi mobilisé pour le passage de valeurs pour le rendu de la vue. L'intérêt est de traiter ces valeurs comme une collection pour en faciliter les traitements via une Map (paires clé/valeur)
- interface `ModelAndView` également mobilisé pour le passage de valeurs pour le rendu de la vue. L'intérêt est de traiter ces valeurs comme une collection d'objets pour en faciliter les traitements via une Map

Exemple avec l'interface Model

```
@RequestMapping(value = "/batiments", method = RequestMethod.GET)
    public String AllBatiments(Model model) {
        List<Batiment> batiments = bs.listeBatiments();
        model.addAttribute("batiments", batiments);
        return "batiments";
    }
```

L'attribut value est un alias pour path. L'annotation plus concise suivante

```
@GetMapping(path = "/batiments")
```

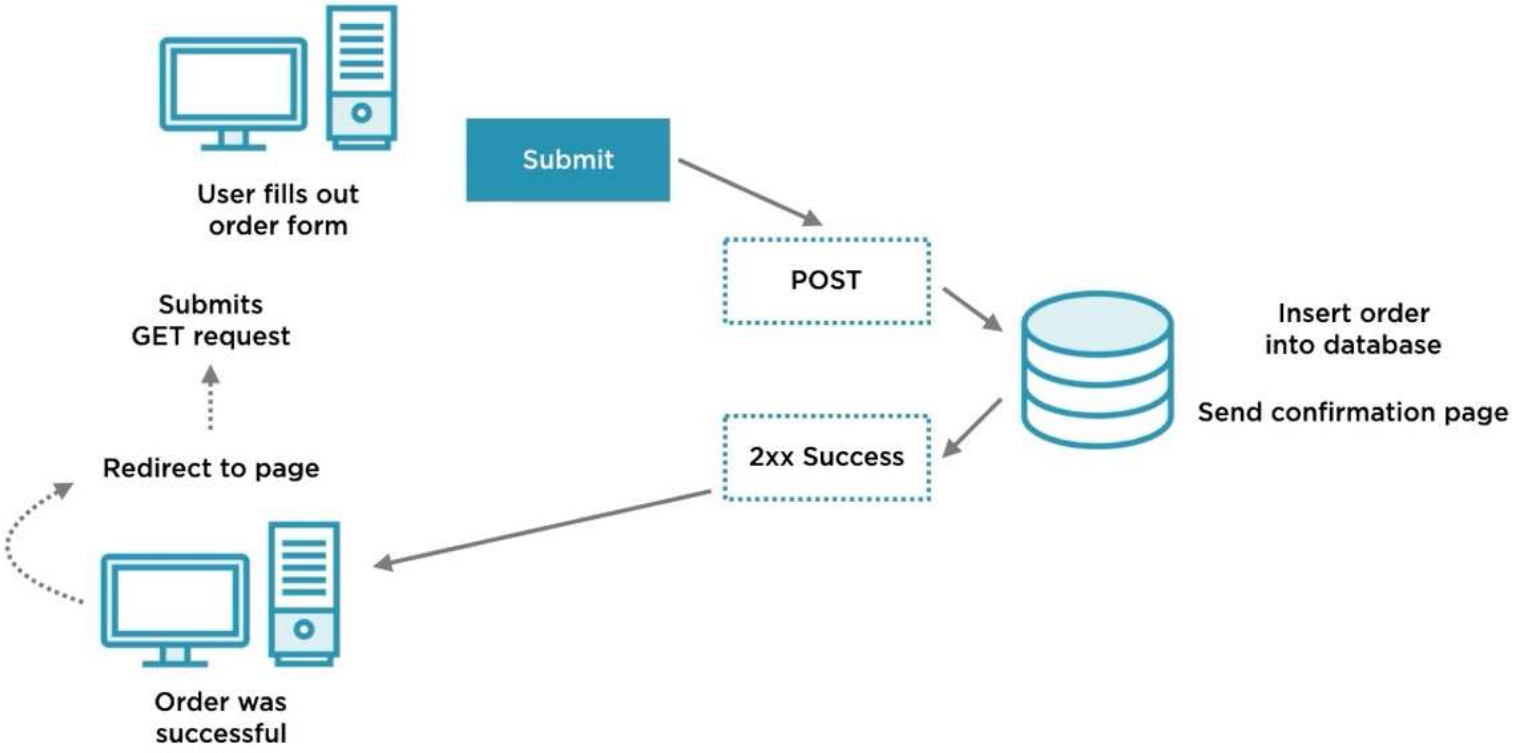
aurait exprimé les mêmes choix

Annotation pour substituer une partie du chemin

```
@GetMapping(path = "/batiments/{codeB}")
public String getBatimentByCode(@PathVariable("codeB") String codeB,
                Model model)
   {
    Optional<Batiment> batiment_opt = bs.batimentParCode(codeB);
    Batiment batiment = batiment_opt.get();
    model.addAttribute("batiment", batiment);
     return "batiment";
    }
```

L'annotation @PathVariable permet de gérer des parties variables dans l'URI de la requête à mapper (ici méthode getBatimentByCode du contrôleur BatimentController)

Redirection : PRG (Post-Redirect-Get) Pattern



- Objectif : après avoir traité un POST, on ne renvoie pas vers la vue d'origine ; on répond par une redirection pour que le navigateur fasse un Get de la vue (données sur la vue effacées)

PRG (Post-Redirect-Get) Pattern -suite-

- Remplacer le "`return`" dans le contrôleur

```
@PostMapping("registration")
public String addRegistration(@ModelAttribute("registration")
        Registration registration) {
  System.out.println("Name: "+registration.getName());
  return "registration";
}
```

- par :

```
@PostMapping("registration")
public String addRegistration(@ModelAttribute("registration")
          Registration registration) {
  System.out.println("Name: "+registration.getName());
  return "redirect:registration";
  }
```

Pourquoi Thymeleaf ?

- Pour beaucoup de développeurs, avoir du JSP (Java Server Pages) dans son application Web est considéré comme une dette technique
- On considère que JSP doit être remplacé par des technologies plus modernes de rendu
- *Thymeleaf* est un framework de templating, qui s'intègre bien avec Spring
- De nos jours, on utilise ce genre de framework en combinaison avec des frameworks de front-end comme Angular, React, Vue, ...

Balises Thymeleaf les plus courantes

L'alias pour l'espace de noms de Thymeleaf est th

label	description
th:text	paramètre de type chaîne de caractères
th:field	champ du modèle
th:object	objet du modèle
th:each	indicateur d'une itération
th:action	prend en charge une action à partir d'un évènement
th:href	référence hypertexte
th:src	pointe sur une ressource
...	...

Expressions à interpréter par Thymeleaf

Un attribut peut se voir affecter une valeur textuelle ou nécessiter une interprétation

forme expression	description
#{...}	expression de message
@{...}	expression d'URL
*{...}	expression de sélection
$&{...}	expressions à évaluer
~{...}	expression d'un fragment

Exemple de balises de la page d'index

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head lang="en">
  <title>Universit&eacute; de Montpellier</title>
</head>
<body>
  <h1> Une vieille Universit&eacute; en Europe </h1>
  <p><img width="100px" th:src="@{images/UM_Logo.png}" /> </p>

  <div>
        <p>Choisissez une page &agrave; consulter :</p>
   <a th:href="@{/campuses}">Voir les Campus en JSON </a><br>
   <a th:href="@{/listeCampus}">Voir les Campus</a>
  </div>
</body>
</html>
```

Création d'un campus (méthodes contrôleur)

```
// pointe sur le formulaire
@GetMapping("/newCampus")
public String showCreateForm(Model model) {
model.addAttribute("campus", new Campus());
return "newCampus";
    }
//en sortie du formulaire
@PostMapping(value = "/processNewCampus")
public String createCampus(@ModelAttribute("newCampus")
          Campus campus) {
cs.createCampus(campus);
return "redirect:/listeCampus";
  }
```

Formulaire associé

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head><title>Ajout Campus</title></head>
<body>
<h1>Ajouter un Nouveau Campus</h1>
<form th:action="@{/processNewCampus}"
    th:object="${campus}" method="post">
    <label>Nom :</label>
    <input type="text" th:field="*{nomC}" required />
    <br />
    <label>Ville :</label>
    <input type="text" th:field="*{ville}" required />
    <br />
    <button type="submit">Ajouter</button>
</form>
</body></html>
```

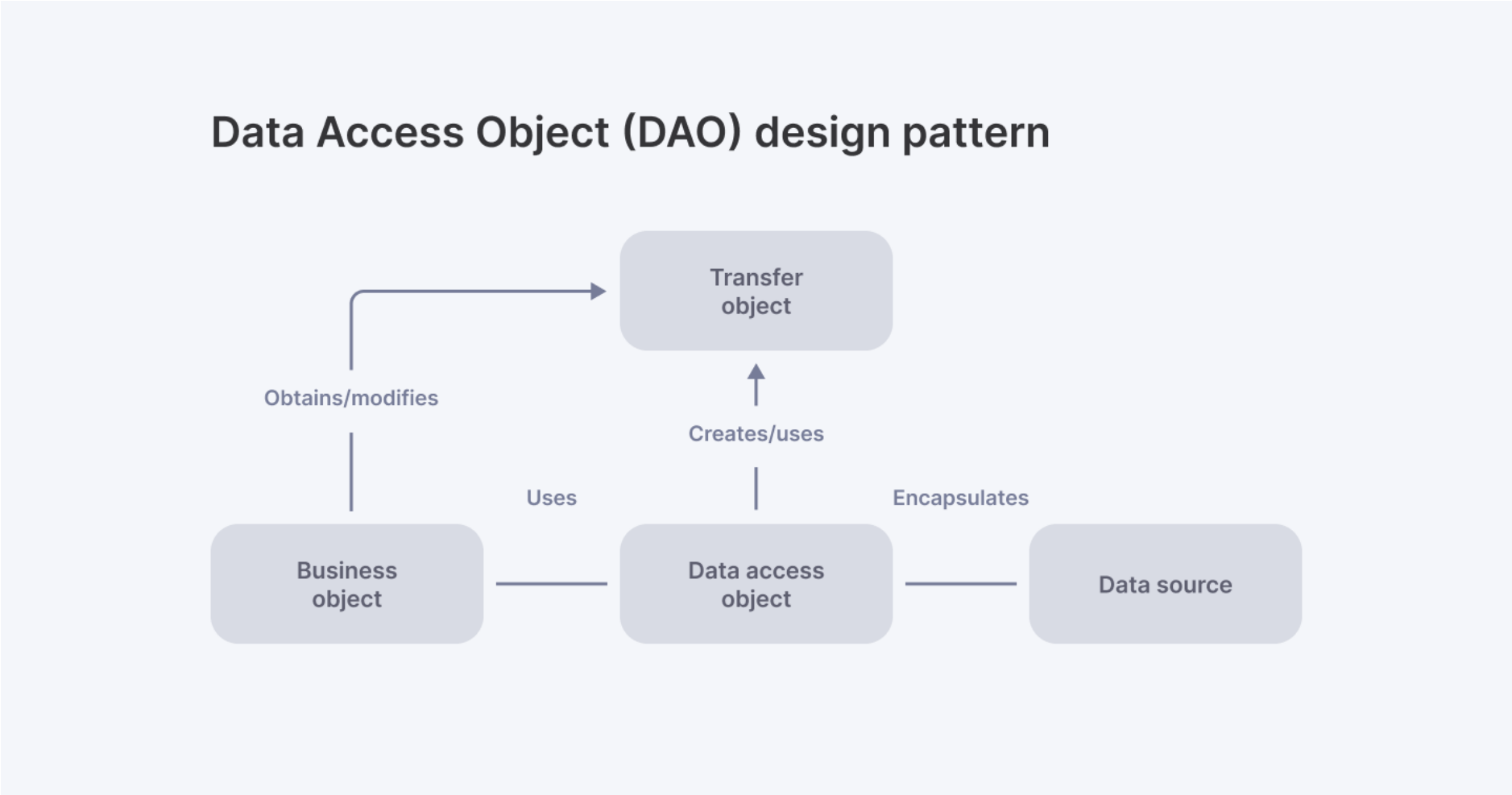
DTO (Data Transfer Object)

Un design pattern pour transférer des données volontairement simplifiées entre les couches applicatives :

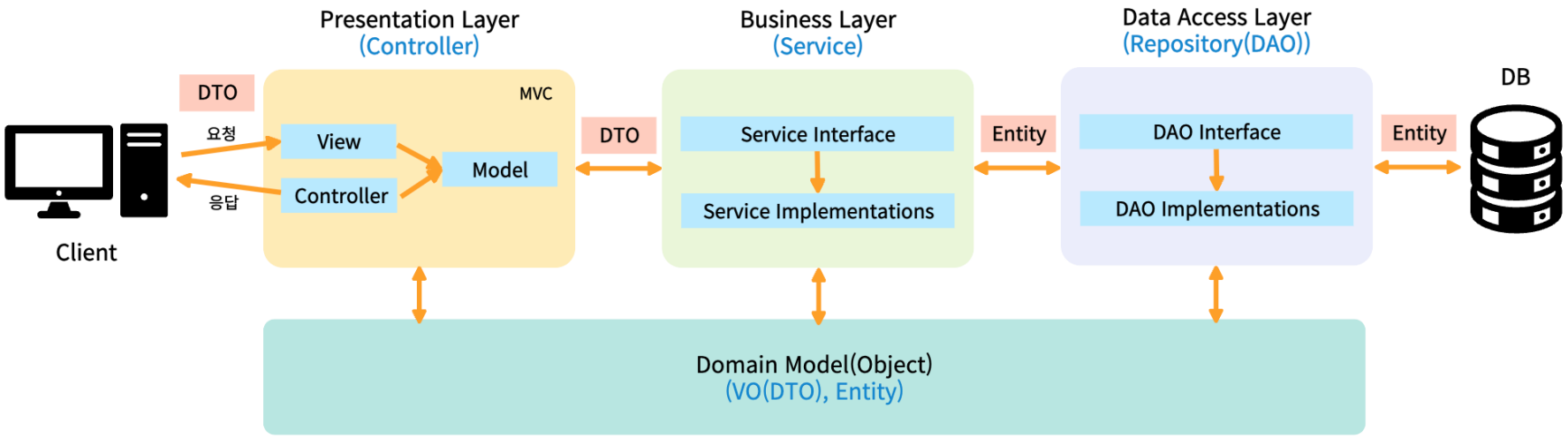
- Immuable : les DTO sont possiblement conçus pour être immuables, ce qui signifie que leurs attributs ne sont pas modifiables une fois valués. caractéristique qui garantit la sécurité du programme et minimise le risque de corruption des données.
- Sérialisable : capacité des DTO à être facilement transformés en un format pouvant être stocké ou communiqué sur un réseau. Cela permet une transmission transparente des données à différents niveaux, de la couche de service à la couche d'affichage.
- Simple : les DTO volontairement dépourvus de logique métier pour viser la simplicité et minimiser le coût du transfert. Utilisation pour le transport de données sans introduire de complexité supplémentaire dans le programme.

UNIVERSITÉ DE MONTPELLIER

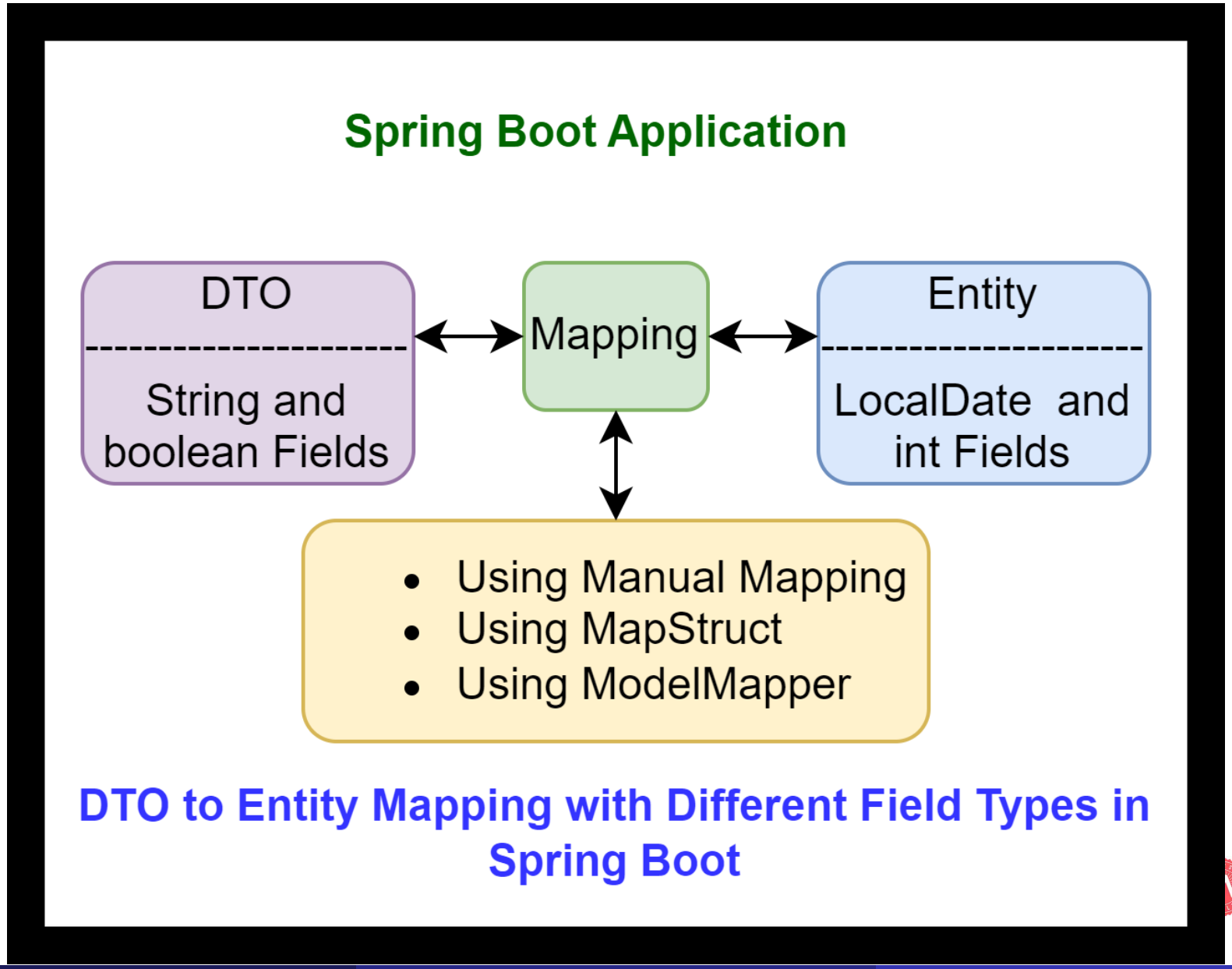
Patterns DTO/DAO



Flux de données de niveau à niveau



Mise en œuvre du mapping



DTO pour Campus

```
public class CampusDTO {
    @JsonProperty("nomCampus")
    private String nomC;
    @JsonProperty("ville")
    private String ville;
    @JsonProperty("batiments")
    private List<BatimentDTO> batiments;
    public String getNomC() {
        return nomC;        }
    public String getVille() {
        return ville;       }
    public void setNomC(String nomC) {
        this.nomC = nomC;       }
    public void setVille(String ville) {
        this.ville = ville;       }
    public List<BatimentDTO> getBatiments() {
        return batiments;       }
    public void setBatiments(List<BatimentDTO> batiments) {
        this.batiments = batiments;       }
}
```

Exemple Interface pour le DTO Mapper

```
package Ex.dto.mapper;
import org.mapstruct.Mapper;

import Ex.dto.dto.BatimentDTO;
import Ex.dto.dto.CampusDTO;
import Ex.modele.Batiment;
import Ex.modele.Campus;

@Mapper(
        componentModel = "spring"
  )
  public interface ICampusMapper {

        CampusDTO campusToCampusDTO(Campus campus);
        BatimentDTO batimentToBatimentDTO(Batiment batiment);

  }
```

Exemple Implementation pour le DTO Mapper

```
@Component
public class CampusMapper implements ICampusMapper {

  @Override
  public BatimentDTO batimentToBatimentDTO(Batiment batiment) {
          if ( batiment == null ) {
              return null;        }
          BatimentDTO btSlimDto = new BatimentDTO();
          btSlimDto.setCodeB( batiment.getCodeB());
          btSlimDto.setAnneeC( batiment.getAnneeC() );
          return btSlimDto;
      }
      ...
      }
```

Mobilisation au sein d'un contrôleur Rest

```
@RestController
@RequestMapping("/sites")
public class CampusDTORestController {
    private ICampusMapper mapstructMapper;
    private CampusRepository cpRepository;
    @Autowired
    public CampusDTORestController(
            ICampusMapper mapstructMapper,
            CampusRepository cpRepository     ) {
        this.mapstructMapper = mapstructMapper;
        this.cpRepository = cpRepository;     }

    @GetMapping("/{id}")
    public ResponseEntity<CampusDTO> getById(
            @PathVariable(value = "id") String id ) {
        return new ResponseEntity<>(
         mapstructMapper.campusToCampusDTO(
                cpRepository.findById(id).get()
                ),
                HttpStatus.OK );
    }
}
```